PROACT

The Complete Manual

Software · Firmware · Python · GUI · ChipWhisperer

A lightweight-crypto RISC-V SoC (ASIC + FPGA)

Version 1.1 – addresses, hardware, C library, host library, GUI and trace capture

SOFTWARE RULE

The PROACT chip is **fabricated and frozen**: the RTL is the read-only, authoritative reference and the software follows it exactly. PROACT is a compact, area-optimized side-channel research SoC; its lean peripherals expect a few simple software access conventions, which the drivers in this repository already implement. Chapter 2 documents those conventions so the behavior is fully understood.

Generated from the maintained sources in this repository (`config/hardware.json`, the firmware/host code, and the Markdown docs).

Contents

CHAPTER 1

Introduction and System Overview

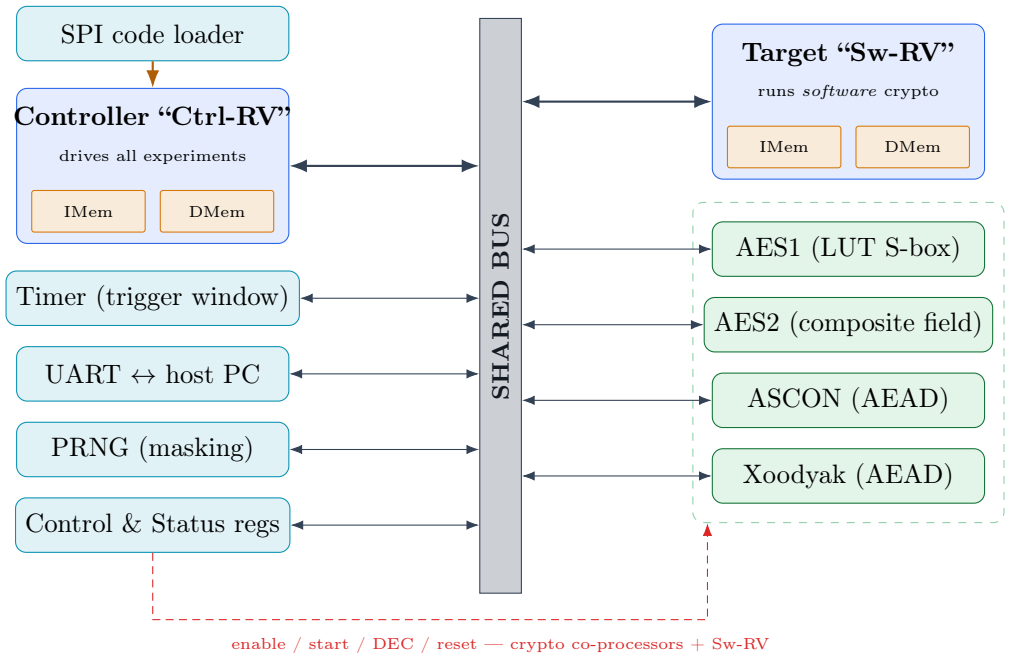
PROACT is a dual-core RISC-V system-on-chip designed for *side-channel research*: measuring the power consumption of cryptographic operations to study their resistance to attacks such as CPA, TVLA and SNR. It was fabricated on the GlobalFoundries 22FDX process (tape-out November 2025) and the same design also runs on FPGA (ChipWhisperer CW305 / PYNQ-Z2).

1.1 System components

Two Ibex RISC-V cores share a bus:

- the **controller** — communicates with the host PC (UART commands, SPI code loading) and drives the crypto cores and the scope trigger;
- the **Sw-RV target** — a second core that runs *software* AES, so hardware and software crypto can be compared on the same silicon.

Four hardware crypto cores are attached to the bus: **AES1**, **AES2** (AES-128, encrypt + decrypt) and **ASCON**, **Xoodyak** (authenticated encryption; the cores implement the encryption datapath and decrypt runs on the host, see Ch. 6). Supporting peripherals: a UART, an SPI code loader, a timer that measures the trigger window, and an LFSR RNG.



HARDWARE FACT (frozen)

For **software**, the ASIC and the FPGA are the *same machine*: the address map is byte-for-byte identical. They differ only in memories, register file, debug pins and clocking — none of which the software can see.

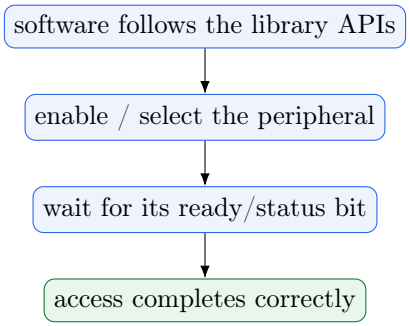
1.2 Software architecture

One JSON file, `config/hardware.json`, is the single source of truth for every address and register bit. `scripts/gen_hardware.py` generates the C header (`proact_regs.h`) and the Python module (`regs.py`) from it, so the firmware, the host tools and this manual cannot disagree. One Python package, `proact_host`, is shared by the command-line tool, the GUI and user scripts.

CHAPTER 2

Design Characteristics and Software Rules

PROACT is a compact, area-optimized side-channel research SoC. To keep the silicon small and the power signatures clean, the bus and peripherals are deliberately minimal — for example the shared bus has no access watchdog: each peripheral answers an access when its data is ready. This is an efficient, common bare-metal design; it simply means the software follows a short list of access conventions. **The drivers and the `proact_host` library already implement every rule below**, so in normal use they require no direct attention. This chapter documents the behavior so it is fully understood and so anyone extending the firmware keeps the same conventions.



SOFTWARE RULE

The access conventions (all handled by the drivers):

1. `0x10007000` (`Co_re`) is a reserved expansion slot with no core attached in this revision — do not access it.
2. Set a crypto core's `ENABLE` bit before accessing its registers; run one core at a time (clean power-gating keeps unused cores quiet).
3. UART writes use offsets `0x00` (data) and `0x04` (baud) — use `putchar()` / `uart_set_baud_divisor()`.
4. UART receive uses the status handshake: poll status-register bit 0 (`STAT_UART_RVALID` = "RX FIFO not empty"), then read the byte. `uart_getchar()` does this.

NOTE

Note — UART TX FIFO depth. The transmit FIFO holds 512 bytes. Output produced faster than the serial line drains it fills the FIFO, so the firmware paces `putchar()` to stay within the line rate (scale the pacing if the baud rate is lowered). This keeps verbose logging lossless.

HARDWARE FACT (frozen)

Global trigger = control bit 30 (`0x40000000`), confirmed by the hardware design team. The control register carries 31 defined bits, so the trigger (the register's top defined bit) is bit 30. The *status* register is a full 32-bit register, and status bit 0 is the UART "RX not-empty" flag while status bit 31 is the Sw-RV "target done" / Sw-RV trigger source.

The full set with the software rule for each is in `docs/hardware_hazards.md` (rules R1–R10; R10 is the AEAD rule — ASCON/Xoodyak decrypt runs on the host, see Ch. 6).

CHAPTER 3

Installation and Setup

3.1 Prerequisites

Purpose	Tool
Build firmware	`riscv32-unknown-elf-gcc` + `srec_cat`
Host control / capture	Python ≥ 3.9 (`requirements.txt`)
USB bridges	MCP2200 (UART) + MCP2210 (SPI), via `hidapi`
Capture (optional)	ChipWhisperer 6 (CW305 + Husky)
RTL simulation (optional)	Icarus Verilog ≥ 11
PDF manual (optional)	`pdflatex` (TeX Live)

3.2 Install the host tools

The "host tools" are the PC-side Python package (`proact_host`) plus the CLI and GUI that run on the host PC and communicate with the PROACT board over USB. Nothing here runs *on* the chip — these tools program the chip, send it commands, read results, and capture power traces. Installing them lets the CLI, the GUI and user scripts share one tested library instead of each re-implementing the serial protocol. The recommended install is one command, followed by *always* launching through the wrapper scripts:

```
bash tools/setup_env.sh # one-time: builds a dedicated venv at ~/.proact-venv
./run_cli.sh info # the CLI -- prints the address map, no board needed
./run_gui.sh # the GUI
```

The wrappers pick the right interpreter automatically. This avoids the common pitfall where a bare `python3` (e.g. a pyenv shim) is a *different* Python that is missing `PyQt6`/`hid`/`mcp2210`/`chipwhisperer`. For a manually managed environment, install `requirements.txt` plus `pip install -e Software/Python[all]` (adds the `proact` CLI entry point).

WARNING

Never run the tools with `sudo`. Running as root *breaks* the ChipWhisperer install (it lives under the invoking user's `~/.local`). Device access comes from the udev rules instead — see the USB permissions section below.

3.3 Build the firmware

The two RISC-V cores run C programs compiled in this step: the *controller* firmware (the command server the host communicates with) and the *target* firmware (software AES on the Sw-RV core). `make` cross-compiles them and produces `.vmem` images that are loaded into the chip's memories.

```
make -C Software/Controller RISCV=riscv32-unknown-elf- # -> main.vmem (one image)
make -C Software/SW_RV RISCV=riscv32-unknown-elf- # -> sw_rv_*.vmem (two)
```

TIP

No local paths are hard-coded. The toolchain prefix is the `RISCV` make variable; override it on the command line. Bench constants (MCP serials, input clock, reset-pin polarity) all live in `proact_host/config.py`.

3.4 USB permissions (Linux) — avoiding `sudo`

The MCP2200 (UART), MCP2210 (SPI) and the ChipWhisperer are USB devices that by default only root can open. Fix this *once* with the provided installer — do **not** work around it with `sudo`:

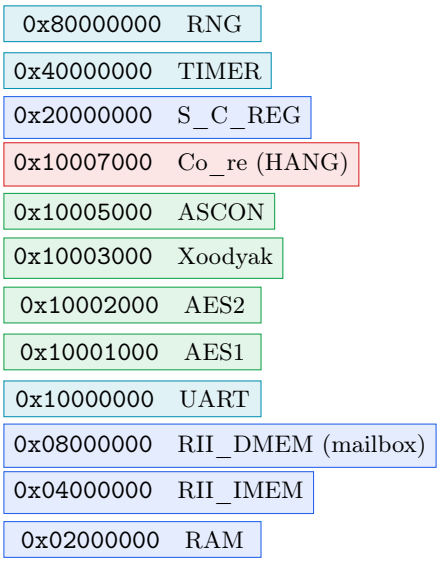
```
sudo bash tools/install_udev.sh # installs udev/60-proact.rules (MCP + all
                                # NewAE devices) and adds you to ‘dialout‘
# then UNPLUG and REPLUG the USB devices (log out/in for the group change)
```

After that, everything runs under the normal user account via `./run_cli.sh` / `./run_gui.sh`.

CHAPTER 4

Memory and Address Map

Every value is generated from `config/hardware.json` and cross-checked against the frozen RTL (34 automated checks pass, `tools/verify_regs_vs_rtl.py`).



Device	Base	Region	Notes
RAM	0x02000000	32 MB	controller data RAM (128 KB phys)
RII_IMEM	0x04000000	32 MB	Sw-RV instruction memory write port
RII_DMEM	0x08000000	128 MB	Sw-RV data mem; controller↔target mailbox
S_C_REG	0x20000000	1 KB	WRITE = control, READ = status (same addr)
UART	0x10000000	1 KB	AHBUART
TIMER	0x40000000	1 KB	trigger-window counter
RNG	0x80000000	1 KB	write = seed; read only by the Sw-RV
AES1	0x10001000	1 KB	AES-128
AES2	0x10002000	1 KB	AES-128 (2nd)
Xoodyak	0x10003000	1 KB	AEAD
ASCON	0x10005000	1 KB	AEAD
Co_re	0x10007000	1 KB	reserved decoder slot (unpopulated)

Unmapped pages `0x10004000` and `0x10006000` return a *clean* decode error. The reserved `Co_re` slot is decoded but unpopulated, so the drivers never address it.

CHAPTER 5

Register Reference

5.1 Control register (write) — 0x20000000

A 31-bit control field. Bits 0–15 are single control bits; bits [22:20] are the trigger-source mux; bit 30 is the capture trigger.

Bit	Field	Bit	Field
0	ENABLE_TARGET	8	START_XOODYAK
1	ENABLE_AES1	9	DEC_XOODYAK
2	START_AES1	10	ENABLE_ASCON
3	DEC_AES1	11	START_ASCON
4	ENABLE_AES2	12	DEC_ASCON
5	START_AES2	13	ENABLE_RNG
6	DEC_AES2	14	ENABLE_TIMER
7	ENABLE_XOODYAK	15	RESET_UART
[22:20]	CFGSEL (trigger source)	29	TRIGGERPC
30	**TRIGGER (0x40000000)**	31	— (31-bit field)

`CFGSEL`: 0 software, 1 ASCON, 2 AES1, 3 AES2, 4 Xoodyak, 5 Sw-RV.

5.2 Status register (read) — 0x20000000

A true 32-bit register. Bits [31:17] are written by the Sw-RV.

Bit	Field	Bit	Field
0	UART_RVALID	9	DONE_ASCON
1	TRIGGER_IN	10	READY_ASCON
2	DONE_AES1	11	TRIGGER_ASCON
3	TRIGGER_AES1	12	TEST_I
4	DONE_AES2	13	AES1_ACTIVE
5	TRIGGER_AES2	14	AES2_ACTIVE
6	DONE_XOODYAK	15	XOODYAK_ACTIVE
7	READY_XOODYAK	16	ASCON_ACTIVE
8	TRIGGER_XOODYAK	31	**TARGET_DONE (live)**

CHAPTER 6

Crypto Cores

6.1 AES1 and AES2 — implementation

PROACT has **two** AES-128 cores, both supporting **encryption and decryption**, at a throughput of **11 clock cycles per block**. They are deliberately built with two different S-box implementations so their power signatures can be compared:

Core	S-box implementation	Purpose
AES1	LUT-based (table lookup)	classic table S-box
AES2	Logic-based, composite-field	area/leakage comparison

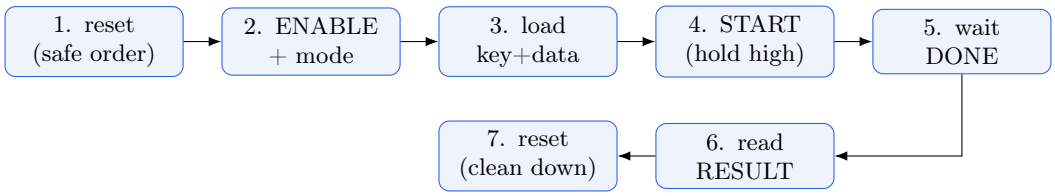
Trigger behavior: the AES trigger is asserted when the `START` signal is received and stays active until the encryption/decryption finishes — so the timer's cycle count is exactly the operation window.

6.2 AES1 / AES2 register offsets

AES-128. Reads *and* writes answer only while the core's `ENABLE` bit is set; `DONE` is valid only while `START` is held high.

Offset	Register	Offset	Register
`0x00`	START (w bit0)	`0x18-0x24`	DATA[127:96..31:0]
`0x04`	DONE (r = ~busy)	`0x28-0x34`	RESULT[127:96..31:0]
`0x08-0x14`	KEY[127:96..31:0]		

The safe AES sequence (encoded in `aes_run()`), verified in simulation against the real core:



6.3 ASCON / Xoodyak offsets

Identical maps (only the base differs). 128-bit KEY and NPUB are shifted in as 4×32-bit writes; AD/PT go into FIFOs; CT/TAG are read back.

Offset	Register	Notes
`0x00`	LEN	`{triggercfg[23:16], pt_len[15:8], ad_len[7:0]}` (byte lengths)
`0x04`	KEY	32b ×4 → 128b
`0x08`	NPUB	32b ×4 → 128b
`0x0C`	AD	write → AD FIFO
`0x10`	PT	write → PT FIFO
`0x14`	CT	read ← CT FIFO
`0x18`	TAG	read ← TAG FIFO

NOTE

AEAD = hardware encrypt + software decrypt. The ASCON and Xoodyak cores implement the *encryption* datapath — the operation side-channel capture measures — and encrypt bit-exactly against the reference vectors. Decryption and tag verification run on the host, so the workflow is: **encrypt on the chip, decrypt + verify the tag on the PC** with `proact_host/aead_soft.py` (bit-exact ASCON-128 v1.2 / Xoodyak v2; decrypt returns `None` on a wrong tag; not constant-time — for validation, not production keys). The firmware's `decrypt=1` path is bounded, so a stray on-chip decrypt attempt returns promptly rather than running. AES1/AES2 decrypt fully in hardware.

The `triggercfg` field — selecting the trigger window

For ASCON and Xoodyak the 7-bit `triggercfg` (LEN bits [23:16]) defines the *trigger window* = [start condition] → [stop condition], so that exactly one phase of the operation can be captured.

Step 1 — mode (`triggercfg[0]`):

- **Mode 0** (`triggercfg[0]=0`): the trigger goes LOW only when `done_o=1` — simplest: start at the chosen state, stop at done.
- **Mode 1** (`triggercfg[0]=1`): the trigger goes LOW at a selected stop state (bits [6:4]).

Activation `triggercfg[3:1]` (trigger HIGH at)		Deactivation `triggercfg[6:4]` (Mode 1, LOW at)	
0	`S_KEY`	0	`tr_kEY_n`
1	`S_NPUB`	1	`tr_n_s`
2	`S_AD`	2	`tr_s_p`
3	`tr_s_p`	3	`S_PT`
4	`S_PT`	4	`S_IDLE` or `done_o`

Example: `0x12` = Mode 0, rise at the nonce (`S_NPUB`), fall on done — the verified default. Use `0x01` to capture the key-absorption phase only. The `AEAD_TRIG(mode,rise,fall)` macro / the GUI "Internal trigger" field build these values.

6.4 PRNG (pseudo-random number generator)

The RNG is a Linear-Feedback Shift Register (LFSR). The **controller** enables/disables and reseeds it; the **Sw-RV target** core reads the generated random values (the natural consumer). Software interface:

```
ctrl_set(CTRL_ENABLE_RNG); ctrl_flush(); // enable (or clear to disable)
proact_write32(0x80000000, 0x12345678); // RNG seed register
// the Sw-RV target reads random words from its data-side RNG address
```

6.5 Timer

The timer is enabled/disabled by the controller and **counts clock cycles while the trigger is high** — so it measures the exact operation window. The trigger is driven through the shared Status & Control registers, which all cores reach. Software interface:

```
ctrl_set(CTRL_ENABLE_TIMER); ctrl_flush(); // enable (or clear to disable)
uint32_t cycles = proact_read32(0x40000000); // read the cycle count
```

CHAPTER 7

The Controller C Library

Include `proact_regs.h` (map) and the driver headers. Every function encodes the safe access sequence for its peripheral.

7.1 Device access + safe UART (`proact_common.h`)

```c
void dev_write(uintptr_t addr, uint32_t val); // 32-bit MMIO write
uint32_t dev_read (uintptr_t addr); // 32-bit MMIO read
int putchar(int c); // paced TX -- cannot overflow the 512B FIFO
int puts(const char *str); // paced string TX
void put_hex(uint32_t h); // 8-digit hex TX
int uart_rx_ready(void); // 1 if a byte is waiting (safe status read)
int uart_getchar(void); // BLOCK then read (never hangs on idle UART)
int uart_getchar_timeout(uint32_t spins);
void uart_set_baud_divisor(uint32_t divisor); // writes UART+0x04 only
void proact_idle_forever(void); // park the core (NOT the old sim_halt!)
```

7.2 Control + status registers

```c
// proact_ctrl.h -- shadow-register model; writes are absolute
void ctrl_reset(void); // shadow = 0
void ctrl_set(uint32_t bits); // shadow |= bits (use CTRL_* macros)
void ctrl_clr(uint32_t bits); // shadow &= ~bits
void ctrl_flush(void); // write shadow to hardware (bit31 masked)
void ctrl_flush_debug(int debug);
// proact_status.h -- status reads always answer, so waiting is safe
uint32_t status_read(void);
int status_test(uint32_t mask);
void status_wait(uint32_t mask); // spin until mask set
int status_wait_timeout(uint32_t mask, uint32_t spins);
```

7.3 Crypto drivers

```c
// proact_aes.h core = PROACT_AES1 | PROACT_AES2
void aes_run(proact_aes_core_t core, const uint32_t key[4],
             const uint32_t data[4], uint32_t out[4], uint32_t decrypt,
             uint32_t verbose); // full safe reset->...->reset sequence
void aes_reset(proact_aes_core_t core);
// proact_aead.h core = PROACT_ASCON | PROACT_XOODYAK
// NOTE: correct for ENCRYPT only -- hardware decrypt cannot complete (Ch.6);
// a decrypt attempt ends at the bounded status_wait_timeout and returns zeros.
void aead_run(proact_aead_core_t core, const uint32_t key[4],
              const uint32_t nonce[4], const uint32_t *ad, const uint32_t *pt,
              uint32_t *ct, uint32_t *tag, uint32_t ad_len, uint32_t pt_len,
              uint32_t decrypt, uint32_t triggercfg, uint32_t verbose);
// proact_aead_kat.h -- on-chip ASCON+Xoodyak encrypt known-answer test
int aead_kat_run(void); // reference vectors; CMD_AEADKAT 0x1A
// proact_experiments.h -- known-answer / round-trip self-tests
int run_all_experiments(uint32_t debug); // returns number of FAILED suites
                                         // (AEAD decrypt half: expected FAIL)
```

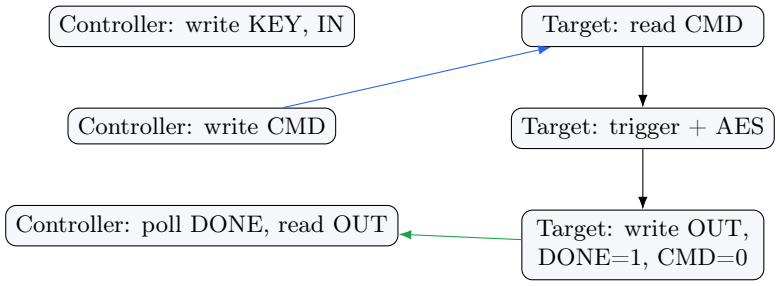
7.4 Sw-RV target driver (proact_swrv.h)

```c
void swrv_enable(void); // release the target from reset (boots it)
void swrv_select_trigger(void); // route the Sw-RV trigger to the scope
void swrv_load_imem_word(uint32_t off, uint32_t word); // load target code
void swrv_load_dmem_word(uint32_t addr, uint32_t word); // load target data
int swrv_aes_block(const uint32_t key[4], const uint32_t in[4],
                   uint32_t out[4], int decrypt, uint32_t timeout);
```

CHAPTER 8

The Target (Sw-RV) Software

The target runs a portable AES-128 (encrypt + decrypt). The controller loads its two images, releases it from reset, and exchanges data through the shared data-memory mailbox — **one command word and one done word per block**, not one handshake per data word (a significant speed-up over the previous flow).



Mailbox addresses (shared, same on both sides): `KEY 0x08003F00`, `IN 0x08003F10`, `OUT 0x08003F40`, `CMD 0x08003F80`, `DONE 0x08003F84`. The Sw-RV cannot reach the UART, so it never prints; it raises the scope trigger by writing status bit 31 (with `cfg_sel`=Sw-RV).

CHAPTER 9

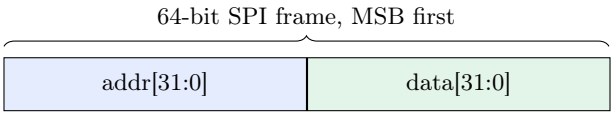
Building and Programming

9.1 Build outputs

The **controller** build emits *one* combined, absolute-addressed, byte-swapped `main.vmem` (`.text` + `.data`/`.rodata` in a single image — the SPI loader routes each word by address), plus an `.elf` and a disassembly. The **Sw-RV** build emits two images (`sw_rv_imem.vmem` + `sw_rv_dmem.vmem`); its data (S-box tables) lives in a loadable section — confirmed present, not zeroed.

9.2 The SPI code loader

The MCP2210 streams 64-bit frames, *address in the upper word*, MSB-first, while driving the reset lines:



Boot order: hold reset low → raise SPI reset → set chip-select → stream → release reset to run. The GUI and `proact program` perform this sequence automatically.

CHAPTER 10

The Python Host Library

`import proact_host`. Hardware backends are imported lazily, so the package imports even without a bench attached.

10.1 Transport — `transport.py`

```
from proact_host.transport import UartTransport, ProactTarget
t = UartTransport(port=None, baud=115200).open() # None = auto-detect MCP2200
chip = ProactTarget(t)
chip.select("aes1"); chip.set_key(key16); chip.set_plaintext(pt16)
chip.set_decrypt(False); chip.enable_sendback()
mode, payload = chip.run_and_read() # -> (mode, 16/32 bytes)
# other methods: set_nonce, set_ad, set_trigger_cfg(0x12), set_cfgsel("aes1"),
# seed_rng, get_timer(), self_test(), load_target_imem/dmem,
# read_status(), write_control(v), aead_kat() -> (xoo_ok, asc_ok)
```

For ASCON/Xoodyak, decryption runs on the host (the cores implement the encryption datapath, Ch. 6). Decrypt on the PC instead:

```
from proact_host import aead_soft # pure-Python, dependency-free
ct, tag = aead_soft.ascon128_encrypt(key, nonce, ad, pt) # bit-exact vs chip
pt2 = aead_soft.ascon128_decrypt(key, nonce, ad, ct, tag) # None on a bad tag
# same API for xoodyak_*; words_to_bytes/bytes_to_words convert chip words
```

10.2 Programmer + resets

```
from proact_host.programmer import Mcp2210Programmer
from proact_host.resets import ResetController
prog = Mcp2210Programmer().open()
prog.program("Software/Controller/main.vmem", progress=print)
rc = ResetController(prog)
rc.apply_mode("global") # none|default_program|controller|global|spi
rc.status() # {line: True(active)/False(reset)/None}
```

10.3 ChipWhisperer capture — `capture.py`

```
from proact_host.capture import ChipWhispererCapture
cap = ChipWhispererCapture(samples=5000, platform="asic") # or "fpga" (CW305)
cap.connect(clock_hz=50e6) # asic: Husky HS2 clkgen; fpga: CW305 PLL + extclk
cap.clock_status() # {platform, target_clock_MHz, adc_freq_MHz, hs2,
                   # clock_source, locked}
cap.arm(); # ... run the op ...; trace = cap.capture(); cap.disconnect()
```

10.4 Experiment, storage, validation, inputs, self-check

```
from proact_host.experiment import PROACTExperiment
with PROACTExperiment(platform="asic", target="aes1", traces=1000,
                      output="experiments/aes1") as exp:
    exp.prepare(); exp.capture(); exp.save() # HDF5 (or NPZ) with metadata
```

```
# also: storage.TraceStore/load, validation.aes128_encrypt_block/validate_aes,
# inputs.InputPlan/Variable/parse_input_file,
# monitor.MonitorDecoder/safe_text (noise-tolerant UART decoding)
```

The single unified live self-check is `proact_host.fullcheck.run_full_check(target, scope=None, ...)` — the same engine behind the GUI's **Self-Check (A–Z)** tab. It sweeps the whole system: UART link + baud, AES1/AES2 encrypt KAT + decrypt round-trip, ASCON/Xoodyak on-chip encrypt KAT (`aead_kat()`) + `aead_soft` software decrypt round-trip, timer, control write, PRNG, Sw-RV software AES, and optional scope clock lock + trace capture. It **passes 100% on the real CW305 bench** and also serves as the ASIC chip-screening tool. (The older `selfcheck.run_self_check` per-core check is superseded by it.)

CHAPTER 11

The Command-Line Interface

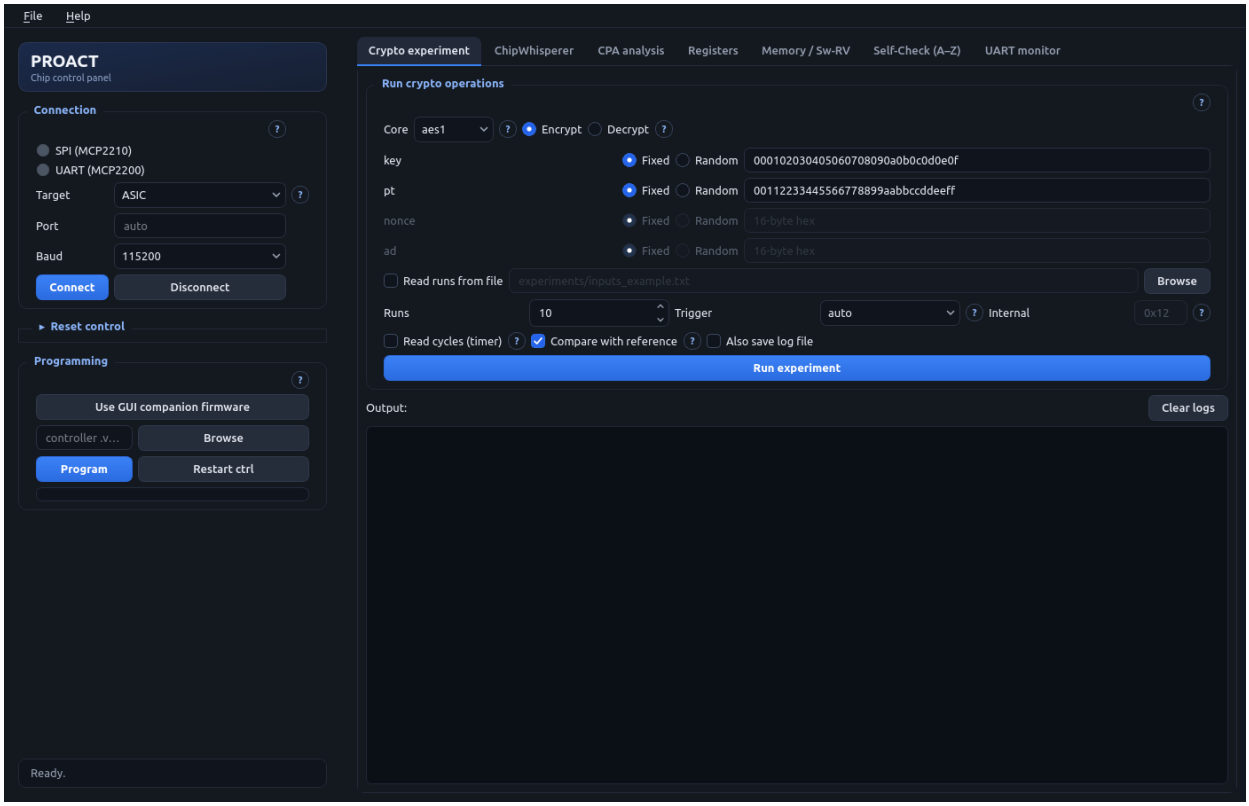
Command	Function
`proact info`	print the address map + bench config
`proact devices`	list detected MCP2200/MCP2210 + ports
`proact build-controller`	build the controller firmware
`proact build-target`	build the Sw-RV firmware
`proact test`	host-side protocol + AES-reference self-check
`proact program -vmem main.vmem`	load firmware over MCP2210 SPI (full reset choreography)
`proact run -core aes1 ...`	run one operation, print the result
`proact capture -core aes1 -traces N`	capture a trace set
`proact selftest`	trigger the *on-chip* self-test (`0x11`; its AEAD decrypt half runs on the host by design, Ch. 6)
`proact load-swrv`	load + start the Sw-RV software-AES target
`proact gui`	launch the GUI

Run the CLI through `./run_cli.sh` (e.g. `./run_cli.sh info`) so the right Python is used — never with `sudo`. The full A–Z *live* self-check is `fullcheck.run_full_check()` / the GUI's Self-Check (A–Z) tab (Ch. 10).

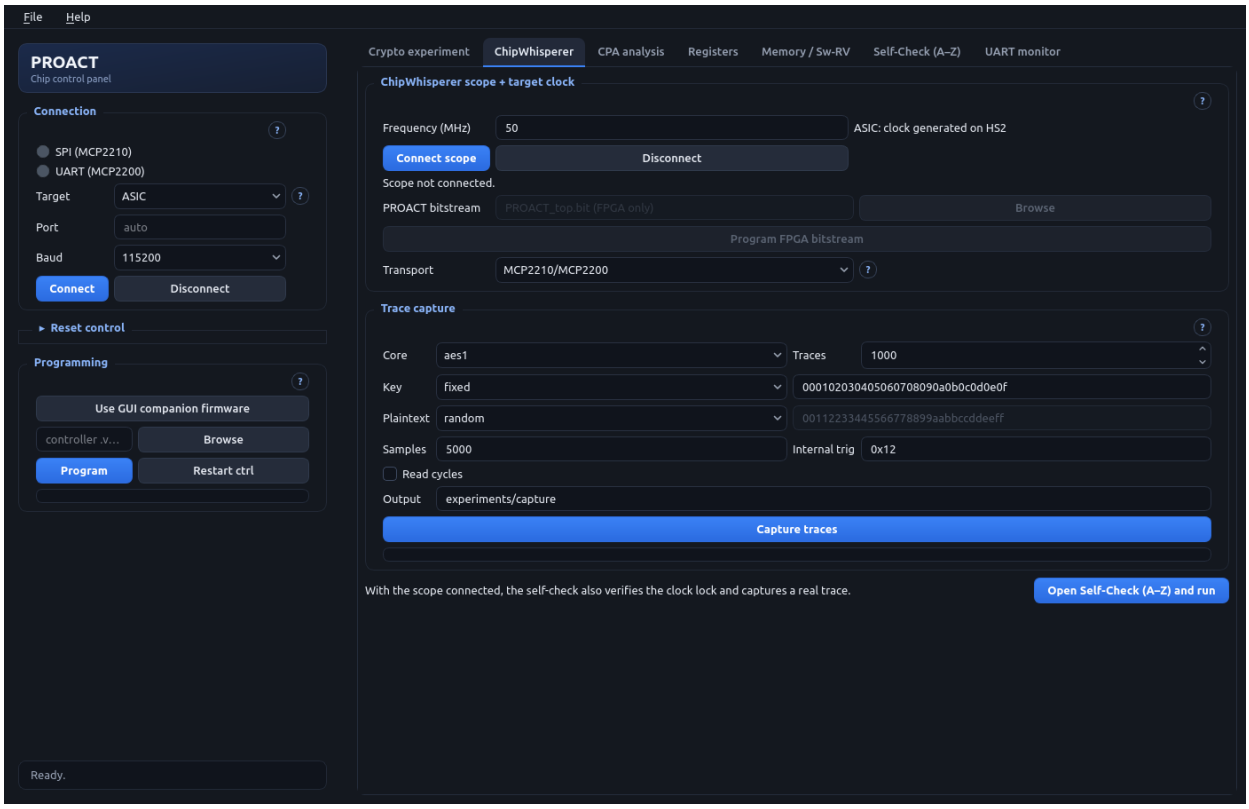
CHAPTER 12

The Graphical User Interface

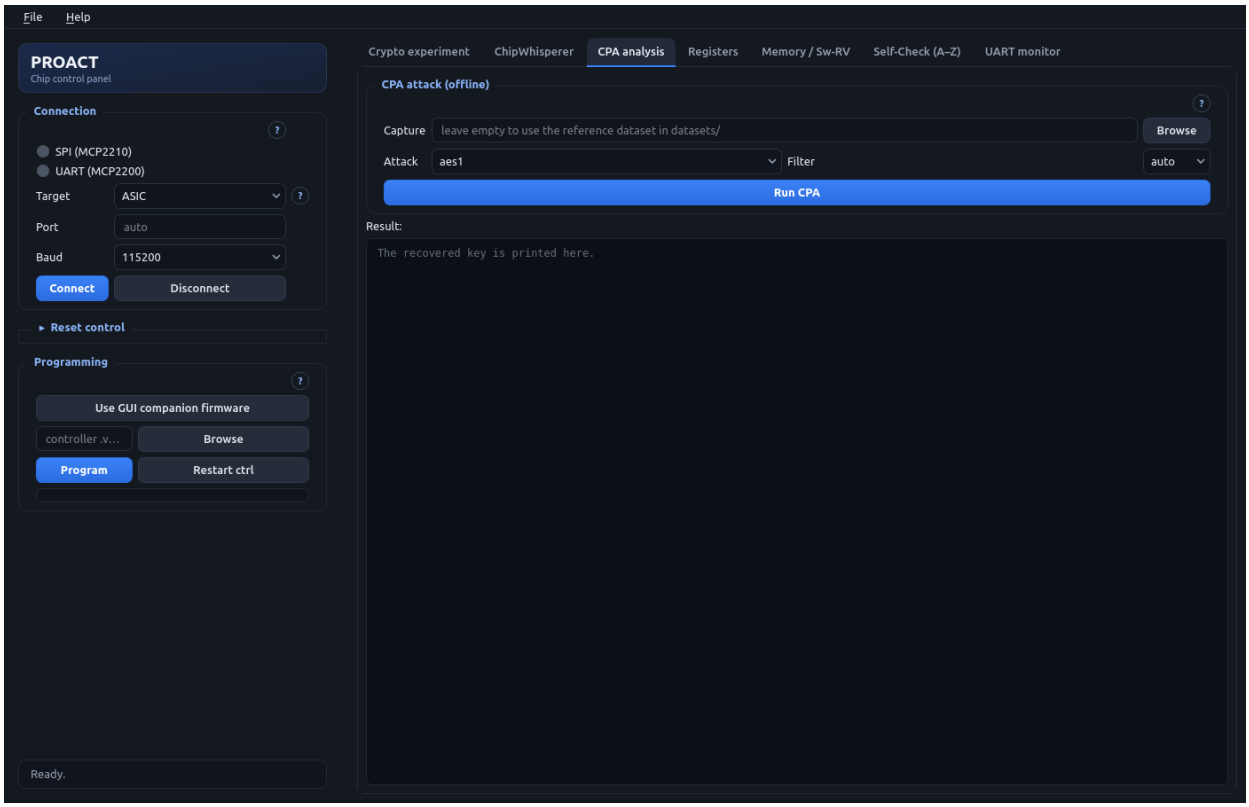
Launch with `./run_gui.sh` — never with `sudo` (root breaks the ChipWhisperer install; device access comes from the udev rules, Ch. 3). The left sidebar is connection, reset control (with LED indicators) and programming: connection and programming are always visible, while reset control is a collapsible panel that starts *collapsed* (click its header to expand it), since it is a bring-up tool rather than a per-run control. The centre holds **seven tabs**, one per workflow step, so no page has to be scrolled: the crypto experiment, ChipWhisperer, **CPA analysis**, registers, **Memory / Sw-RV**, **Self-Check (A–Z)** and the UART monitor. Every panel has a **(?)** help button.



Crypto experiment: pick a core (nonce/AD auto-disable for AES), choose encrypt/decrypt, set each input Fixed or Random (or read a whole run list from a text file), set the number of runs, the trigger source and the internal (ASCON/Xoodyak) trigger phase, optionally read the cycle count, and compare each result with a software reference. Output goes to the window and optionally a log. For ASCON/Xoodyak, decryption runs on the host with `aead_soft` (Ch. 6) — the cores implement the encryption datapath — so a *Decrypt* attempt there ends at the firmware's bounded timeout and returns zeros. AES1/AES2 decrypt normally.

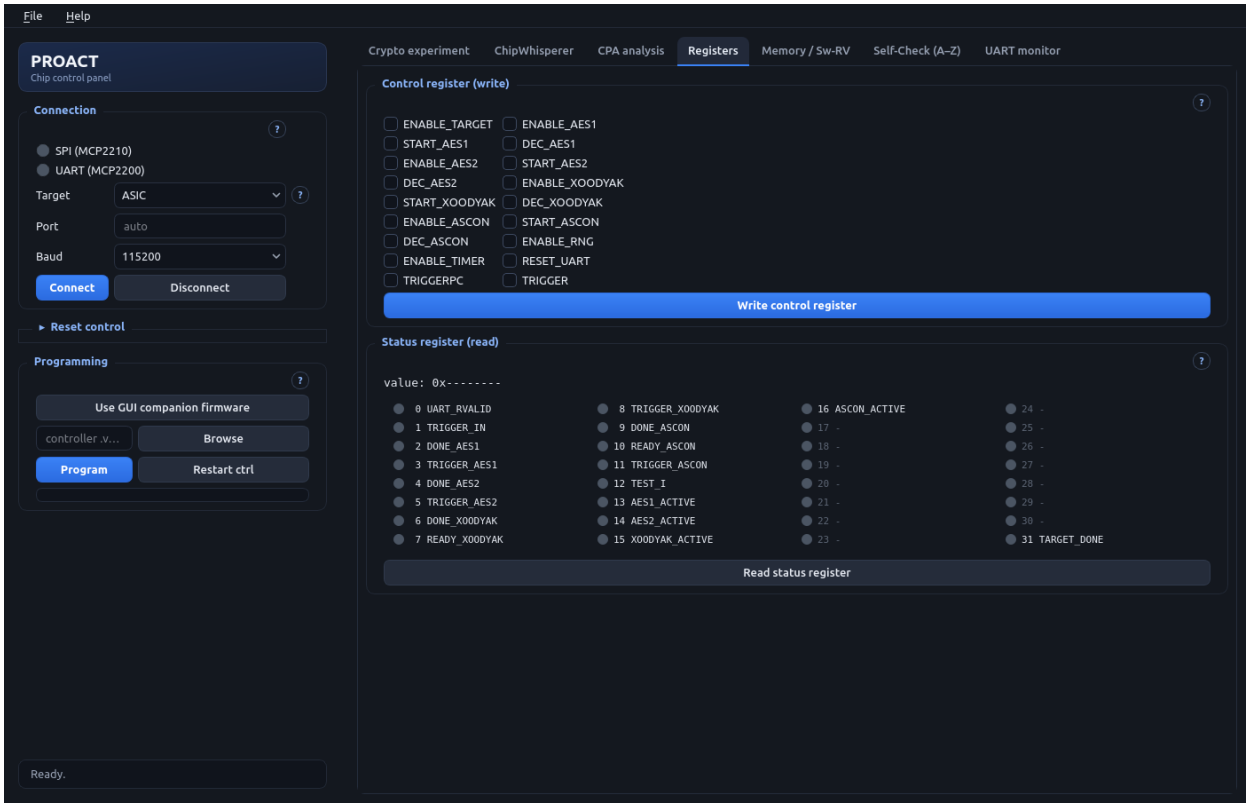


ChipWhisperer: connect/disconnect the Husky, set the target clock (ASIC: 50 MHz on HS2; FPGA: program the CW305 bitstream, on-board PLL), choose the transport (MCP bridges or the Husky's own SPI/UART with GPIO3 chip-select), and capture traces. There is no separate self-check panel here: one compact row at the bottom of the page — a one-line reminder plus the single button *Open Self-Check (A–Z) and run* — jumps to the Self-Check tab and runs it.

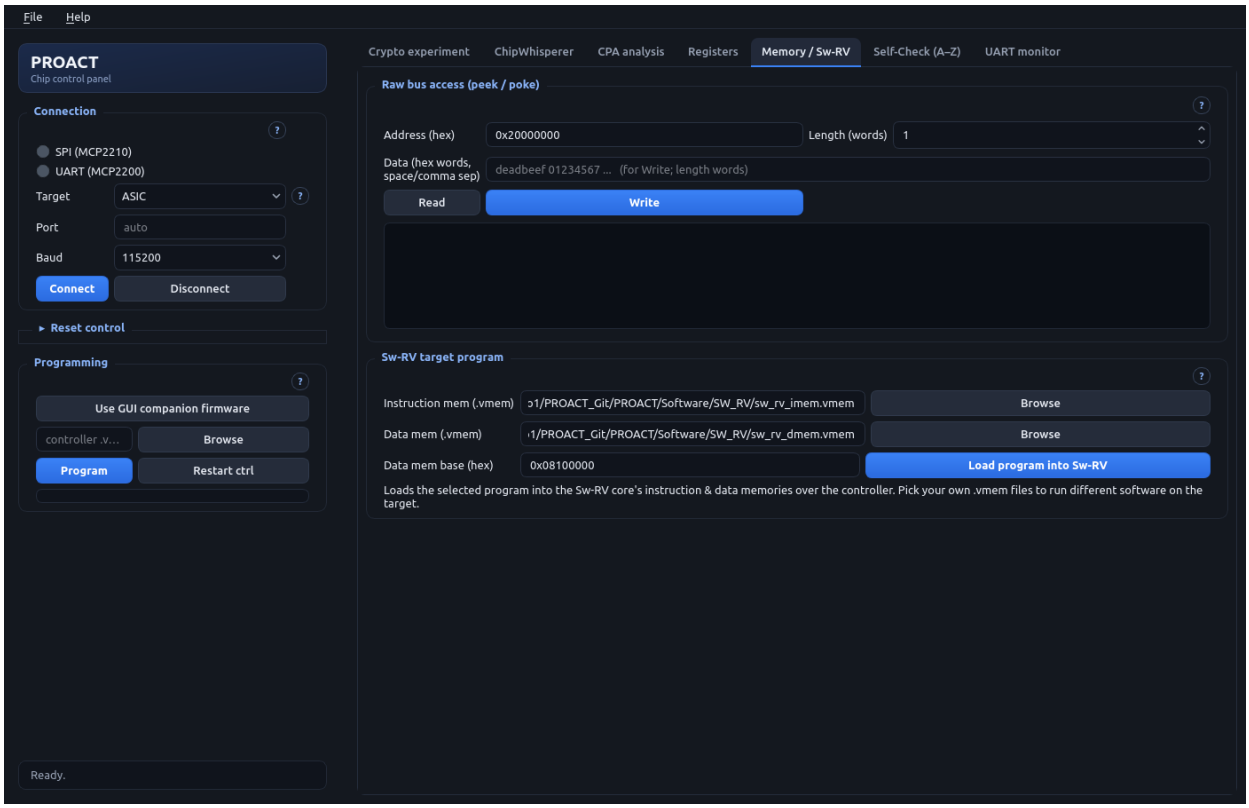


CPA analysis: the offline correlation power-analysis attack, on its own page. Leave *Capture*

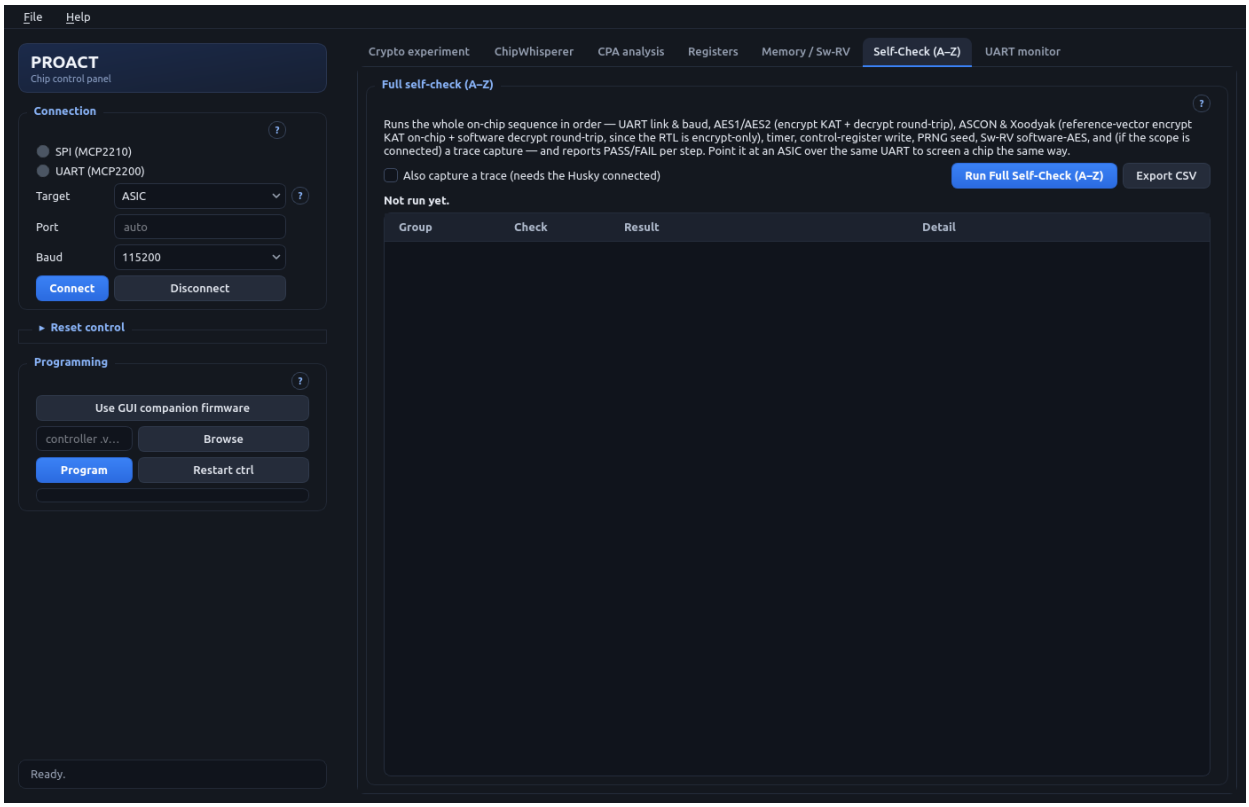
empty to attack the reference traces shipped in `datasets/`, or browse to your own `.npz`/`.h5`; pick the leakage model (`aes1`/`aes2` last-round ciphertext, `swrv` first-round S-box) and the moving-average filter width (*auto* picks it from the data; measured best AES1 8, AES2 2, Sw-RV 16), then *Run CPA*. It needs no hardware, so it doubles as a no-board demo; the recovered key is printed in the console that fills the rest of the page.



Registers: tick control-register bits and write them; read the status register and see each set bit light up with its meaning in a grid of 8 rows by 4 columns.



Memory / Sw-RV: the two bring-up and debug panels. *Raw bus access* peeks and pokes any bus address directly through the controller (32-bit words, stepping 4 bytes) — reads are safe on mapped addresses, but writing to CPU RAM can wedge the chip, so poke only known addresses. *Sw-RV target program* loads the instruction and data `.vmem` images (defaults in `Software/SW_RV/`) at the data-mem base `0x08100000` while the target is held in reset, then releases it so the fresh program boots; afterwards select core `swrv` in the crypto experiment tab to run it.



Self-Check (A–Z): the single unified live check (`fullcheck.run_full_check`, Ch. 10) with a live PASS/FAIL/SKIP table and CSV export — passes 100% on the real CW305; also the ASIC screening tool.

CHAPTER 13

ChipWhisperer and Trace Capture

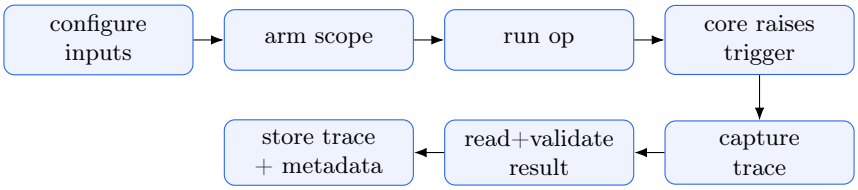
13.1 Choosing the target: ASIC or FPGA

The same PROACT design and the same UART/SPI software run on two platforms, but the **clock** and **programming** differ — pick the target in the GUI's Connection panel (or set `platform` in `ChipWhispererCapture`):

Platform	Clock	Program
ASIC	the Husky generates the target clock on HS2 (`clkgen`, default 50 MHz)	load the controller firmware `.vmem` over MCP2210 SPI
FPGA (CW305)	HS2 disabled; the CW305's on-board PLL provides the clock (output 1, 50 MHz)	first upload the PROACT bitstream (`cw.target(scope, cw.targets.CW305, bsfile=..., fpga_id="100t")`, VCCINT 1.0 V), then load the firmware

The GUI's ChipWhisperer tab exposes the frequency (ASIC) and the bitstream + "Program FPGA bitstream" button (FPGA). The **GUI companion firmware** — the controller command server (`Software/Controller/main.vmem`) — is the on-chip program that understands all the GUI's commands; the Programming panel has a one-click button to select it.

The capture order matches the verified PROACT trigger sequence:



Set the target clock on HS2 (default 50 MHz), select the trigger source to match the core being captured (the firmware auto-sets `cfg_sel`), arm, run, and the scope captures the trigger window. Traces plus full metadata (key, plaintext, output, scope + trigger settings, timestamp) are saved incrementally so a long run survives interruption.

CHAPTER 14

Experiments and Data Storage

Datasets are stored in HDF5 (rich, one file) or NumPy `.npz` (fallback). Each stores traces, plaintexts, keys, outputs, validation flags, a failed-capture log, and metadata (target, platform, scope/trigger settings, timestamp, configuration). Input files (`experiments/inputs_example.txt`) list one run per line as `key=..` `pt=..` `nonce=..` `ad=..` (hex).

CHAPTER 15

Testing and Verification Status

NOTE

The FPGA build is now **bench-verified end to end**: on a real CW305 + Husky the unified A–Z self-check passes 100% (16 pass, 0 fail, 0 skip; 2026-08-07). The fabricated **ASIC has not been screened yet** — the same A–Z check is the screening tool. Every claim below states exactly how it was verified.

Layer	Status
`proact_regs.h` vs frozen RTL	**RTL cross-checked** — 34/34 constants agree
AES driver register sequence	**RTL-simulated** — reproduces the known-answer ciphertext against the real `aes_core` (iverilog)
Controller + target firmware	**builds clean** (0 warnings); one combined controller vmem, two Sw-RV vmem images
Host protocol + AES reference	**unit-tested** (byte-stream + FIPS-197 vector)
GUI	**constructs + renders** headless
On-chip AEAD encrypt KAT	**hardware** — ASCON + Xoodyak both PASS on the CW305 (`CMD_AEADKAT`)
On real FPGA (CW305 + Husky)	**hardware** — full A–Z self-check passes 100%, incl. scope lock + trace capture
AEAD decrypt	**runs on the host by design** — `aead_soft`, bit-exact round trip (Ch. 6)
On the fabricated ASIC	**not yet run** — screened with the same A–Z check

Run all offline checks: `RTL_ROOT=/path/to/ASIC/rtl bash tools/verify_all.sh`. Run the live A–Z check: the GUI **Self-Check (A–Z)** tab or `proact_host.fullcheck.run_full_check()`.

CHAPTER 16

Troubleshooting

Symptom	Likely cause	Fix
CPU waits on an access	accessed a disabled core / reserved `Co_re` slot / read UART before a byte arrived	use the library sequences (Ch.2); they wait on the right ready bit
Output stops mid-print	TX FIFO filled faster than the line drains (not a real freeze)	keep `putchar` pacing; scale it if the baud rate is lowered
No MCP device found	udev rules / wrong serial	install rules (Ch.3); set serials in `config.py`
Wrong/garbled UART text	wrong baud vs input clock	set the clock in `config.py`
Trigger never fires	wrong `cfg_sel` (source) for the running core	use `CTRL_TRIGGER`=bit30; select the trigger source; Sw-RV uses status bit 31
Build error	toolchain not found	pass `RISCV=` to make (Ch.3)
On-chip AEAD decrypt returns all zeros	by design — AEAD decrypt runs on the host (cores implement the encryption datapath, Ch.6)	decrypt + tag-verify on the PC: `proact_host/aead_soft.py`
Tools fail to start / modules missing / "chipwhisperer is not installed"	bare `python3` is the wrong interpreter (pyenv), or running as `sudo`	always launch `./run_gui.sh` / `./run_cli.sh`; never `sudo` — udev rules give device access (Ch.3)

CHAPTER A

Chip Pinout

The fabricated PROACT chip has 28 pins (package view: top, notch up). Reset lines are active-low; the `out_pins[*]` are debug/test outputs (several double as trigger/status probes). Two useful bring-up details from the package:

- **The reset inputs have internal pull-ups** (`B_RST_N`, `spi_global_RST_N`, `C_RST_N`, `spi_c_RST_N` — "IN · PU") enabled on silicon, so they idle at 3.3 V (de-asserted) when left open.
- **Four pins were reassigned to power during packaging** vs the RTL pad list: pins 15 and 28 are `VDD` 0.8 V core, pin 22 is `VDDIO` 3.3 V, and pin 20 is `spi_c_RST_N` — the table below is the *as-packaged* assignment (the wiring reference for the bench). The hardware design team's package drawing can be placed at `docs/images/pinout.png`.

#	Name	Dir	Category	Description
1	`B_RST_N`	In·PU	Reset	board/push-button global reset (active-low)
2	`TX`	Out	UART	UART transmit
3	`RX`	In	UART	UART receive
4	`spi_global_RST_N`	In·PU	Reset	SPI global reset (active-low)
5	`C_RST_N`	In·PU	Reset	controller reset (active-low)
6	`out_pins[0]`	Out	Debug	`data_addr_spi[23]` (SPI mem select)
7	`VDDIO`	Pwr	Power	3.3 V I/O supply
8	`VSS`	Pwr	Power	ground
9	`SYSCLK_P`	In	Clock	main system clock input
10	`spare_io`	In	Debug/Status	mapped to `status_reg[1]`
11	`out_pins[5]`	Out	Debug/Status	`status_bus.uart_rvalid`
12	`out_pins[6]`	Out	Debug	`dev1_req[RII_data/in_mem]`
13	`out_pins[1]`	Out	Debug/Trigger	`trigger_cfg`
14	`trigger_Out`	Out	Trigger	**global trigger output**
15	`Vcore`	Pwr	Power	core supply (0.8 V)
16	`out_pins[7]`	Out	Debug	`flash_dout` (heartbeat LED)
17	`SIn`	In	SPI	SPI serial data in (MOSI)
18	`SSel_n`	In	SPI	SPI chip select (active-low)
19	`sck`	In	SPI	SPI serial clock
20	`spi_c_RST_N`	In·PU	Reset	SPI controller reset (active-low)
21	`out_pins[11]`	Out	Debug	AES1/AES2/Xoodyak/ASCON request
22	`VDDIO`	Pwr	Power	3.3 V I/O supply
23	`trigger_in`	In	Trigger	external trigger input
24	`out_pins[4]`	Out	Debug	`SW_RV pc_out[4]`
25	`out_pins[3]`	Out	Debug	`SW_RV pc_out[3]`
26	`out_pins[2]`	Out	Debug	`SW_RV pc_out[2]`
27	`out_pins[8]`	Out	Debug/Trigger	`trigger_reserve` (Ctrl-RV trigger)
28	`Vcore`	Pwr	Power	core supply (0.8 V)

CHAPTER B

Quick Reference

B.1 UART command bytes

`0x01` KEY · `0x02` PT · `0x03` send-back · `0x04` debug · `0x05` run · `0x06` AES2 · `0x07` Xoodyak · `0x08` ASCON · `0x09` AES1 · `0x0A` arm · `0x0B` NONCE · `0x0C` AD · `0x0D` dec · `0x0E` seed · `0x0F` trig-cfg · `0x10` timer · `0x11` self-test · `0x12` load-imem · `0x13` load-dmem · `0x14` Sw-RV · `0x15` cfg-sel · `0x16` read-status · `0x17` write-ctrl · `0x18` poke · `0x19` peek · `0x1A` AEAD KAT. Result frame: `0xA5 <mode> <len> <payload>`.

B.2 Summary of rules

Trigger = bit 30. Enable a core before access. One core at a time. Leave the reserved `0x10007000` slot alone. Read the UART when RX is ready. ASCON/Xoodyak decrypt on the host (`aead_soft`). Launch via `./run_gui.sh` / `./run_cli.sh` — never `sudo`. Edit `hardware.json`, not the generated files.